

INSTANT — Energy Monitoring Website

A Beginner's Guide to the Components, Operation, Theory, and Code

Version: 1.0 **Date:** August 2026 **Audience:** Beginners — no prior experience with React, electronics, or cloud platforms is assumed. **Subject:** The INSTANT project — a web application that monitors electrical energy in real time and lets you control a relay from any browser.

Table of Contents

1. Introduction
2. Before We Start — The Big Picture
3. Electrical Theory You Need
4. IoT and Web Concepts You Need
5. System Architecture at a Glance
6. Hardware Components
7. The ThingSpeak Cloud Platform
8. How the Website Is Operated (User Guide)
9. The React Basics You Need to Read the Code
10. Project Structure and File Map
11. Application Startup — `index.js` and `index.html`

12. The App Shell — `App.js`
 13. Global State and the Polling Engine — `AppContext.js`
 14. Talking to ThingSpeak — `api/client.js`
 15. Reusable UI Components
 16. The Pages
 17. Theming and Styling
 18. End-to-End Feature Walkthroughs
 19. Error Handling and Edge Cases
 20. Security Considerations
 21. Building, Running, and Deploying
 22. The Android App (APK)
 23. The ESP32 Firmware
 24. Troubleshooting
 25. Glossary
 26. Further Reading
-

1. Introduction

INSTANT is a real-time energy-monitoring web application. It shows live electrical measurements — voltage, current, power, and power factor — on a modern, animated

dashboard, records history over time, and lets you switch a relay (for example, to turn a load on or off) with a single click from any browser.

The project is built from three main pieces:

1. **A hardware device** based on the ESP32S2 microcontroller, which measures the electricity flowing through a cable using non-invasive sensors and sends the readings to the internet.
2. **A cloud database** (ThingSpeak) that stores the readings and the settings, acting as a bridge between the device and the browser.
3. **A website** (the “frontend”) written in React that runs entirely in the browser, reads data from the cloud, and renders it as gauges, charts, tables, and switches.

This report explains each of these pieces. It is written for someone who is comfortable using a computer but has never programmed before. Every chapter builds on the previous one, and all technical jargon is defined in the Glossary at the end.

The goals of this report are to answer four questions:

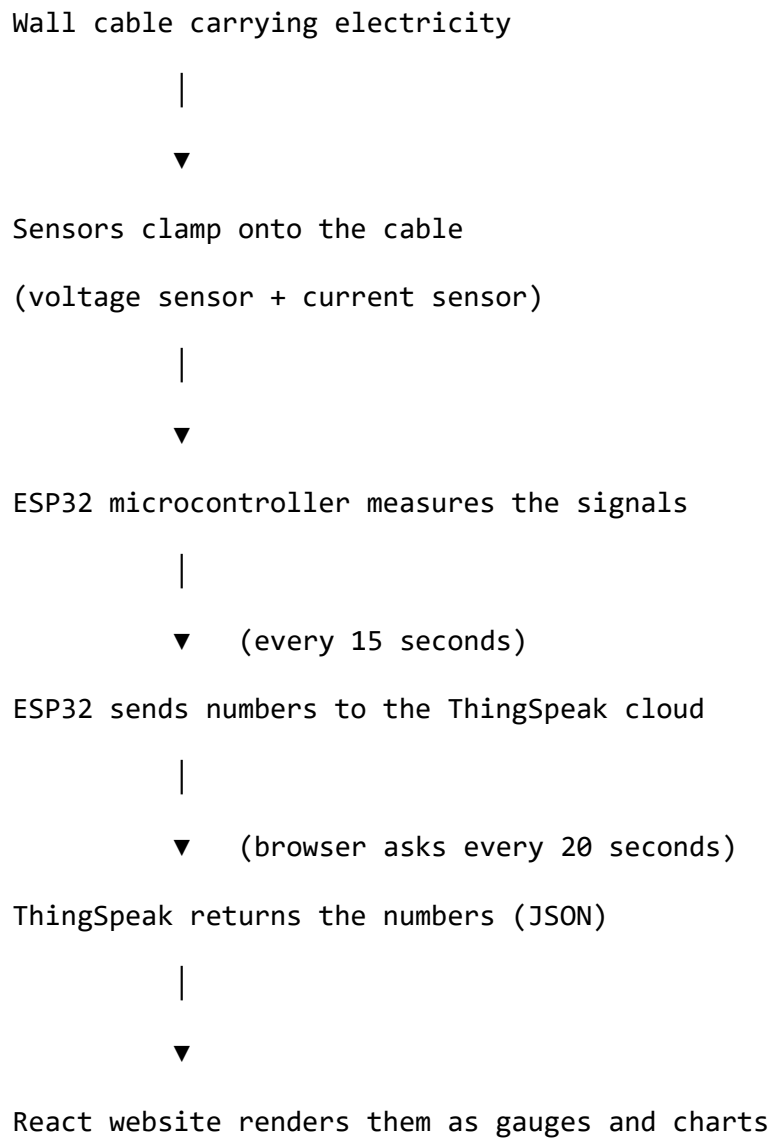
- **What** are the components of the system?
- **How** does the system operate, from sensor to screen?
- **What** is the theory — the electrical and web concepts behind it?
- **How** does the code function, file by file?

By the end, you should be able to look at the project files, follow the flow of data through the system, and explain why the website behaves the way it does.

2. Before We Start — The Big Picture

Imagine you want to know how much electricity a machine is using, and you also want to be able to switch that machine off from your phone, from anywhere in the world. That is exactly the problem INSTANT solves.

Here is the journey of one measurement, from the physical world to your screen:



The same cloud also carries commands in the **opposite** direction:

You click the relay switch on the website

|

▼ (one HTTP request)

Website writes "ON" or "OFF" to ThingSpeak

|

▼ (device checks every 10 seconds)

ESP32 reads the new value and switches the relay

Notice an important design decision: **the website and the device never talk to each other directly**. They both talk to ThingSpeak, which stores data and acts as a middleman. This is a common pattern called **cloud-based communication**, and it means the device and the website do not need to know where the other one is. The device only needs to reach the cloud; the browser only needs to reach the cloud.

Let us look at the same idea from a “roles” point of view:

Piece	Role	Example
Sensor layer	Measures the physical world	ZMPT101B voltage sensor, SCT-013 current sensor
Device layer	Reads sensors, does math, talks to the cloud	ESP32S2 running Arduino firmware
Cloud layer	Stores data, passes messages, handles timing	ThingSpeak channels

Piece	Role	Example
Presentation layer	Shows data, accepts user actions	React website on GitHub Pages

The rest of this report fills in the details of each layer.

3. Electrical Theory You Need

You do not need an engineering degree to understand this project, but a small amount of electrical theory will make everything else clearer.

3.1 Voltage, Current, and Power

- **Voltage (V)** is the “pressure” that pushes electricity along a wire. It is measured in **volts (V)**. In most homes the mains supply is about 220–240 V.
- **Current (I)** is the amount of electricity flowing. It is measured in **amperes** or **amps (A)**. A phone charger might draw about 1 A; a kettle might draw about 10 A.
- **Power (P)** is the rate at which energy is used. It is measured in **watts (W)**. Power is simply the product of voltage and current:

$$P = V \times I$$

So a device running at 230 V drawing 2 A uses 460 W.

3.2 Power Factor (PF)

Real-world devices often do not use the power that they *seem* to use. The apparent power is $V \times I$, but the *actually useful* power (the real power that heats, moves, or charges things) is

smaller, because current and voltage are not perfectly in step (in phase). The ratio of real power to apparent power is called the **power factor**:

$$\text{PF} = \frac{\text{Real Power}}{V \times I}$$

Power factor is a number between 0 and 1. A value close to 1 means the device uses power efficiently. Lighting and heaters have a high power factor; motors and transformers often have a lower one. INSTANT derives PF in the browser as $\text{power} / (\text{voltage} \times \text{current})$ because the device does not send it directly.

3.3 Why We Measure, Not Guess

Instead of cutting the cable and inserting a meter, INSTANT uses **non-invasive sensors** that clamp around the outside of a wire:

- A **voltage sensor** measures the voltage at the terminals (in this project, a ZMPT101B which is a small voltage transformer).
- A **current sensor** is a clamp that measures the magnetic field around the wire (an SCT-013 “split core” current transformer). This is what makes the system safe to install — the live wire is never broken.

The microcontroller reads the two signals many times per second and calculates the RMS (root-mean-square) values of voltage and current. RMS is the standard way to describe an alternating (AC) signal: it is the equivalent steady DC value that would produce the same heating effect.

3.4 Safety Limits

Because electricity is dangerous, the system has safety thresholds:

- **vMax** — maximum allowed voltage,
- **iMax** — maximum allowed current,
- **pMax** — maximum allowed power.

If any live reading goes above its threshold, the website shows a red **FAULT DETECTED** banner, and the device firmware is designed to switch the relay off for safety. The thresholds themselves live in the cloud (the config channel), so you can change them from the website without touching the device.

4. IoT and Web Concepts You Need

INSTANT is an **IoT** (Internet of Things) application: physical things (the ESP32, sensors, relay) connected to the Internet. Several general web concepts are used throughout, so let us define them.

4.1 HTTP and REST

The **HTTP** protocol is how browsers and devices ask for data on the web. A client (browser or device) sends a **request** to a server and receives a **response**. Two verbs matter here:

- **GET** — “please give me this data” (a read).
- **POST** — “please store this data” (a write).

REST is a style of organizing these requests around resources (for example, “the latest reading” or “the settings”). ThingSpeak exposes a REST API: each channel has URLs you can GET or POST to.

4.2 JSON

JSON (JavaScript Object Notation) is a human-readable text format for structured data. Here is a JSON object representing one reading:

```
{  
  "field1": "230.4",  
  "field2": "1.52",  
  "field3": "350.2",  
  "created_at": "2026-08-15T10:30:00Z"  
}
```

JSON is made of pairs of names and values, using braces { } for objects and brackets [] for lists. JavaScript (the language of the website) can turn JSON into real objects with one function call.

4.3 CORS (Cross-Origin Resource Sharing)

Normally, a web page at one address is not allowed to fetch data from another address — a security rule called the **same-origin policy**. **CORS** is the mechanism by which a server says “yes, I allow pages from other origins to read my data.” ThingSpeak sends `Access-Control-Allow-Origin: *` on all its endpoints, which means *any* website (or even a file on your disk) can read and write it directly. This single decision is why **INSTANT** needs **no backend server**: the browser talks straight to ThingSpeak.

4.4 Polling

Because the browser cannot receive data that the server decides to push, it must ask repeatedly. This is called **polling**: on a timer, ask “is there anything new?” In INSTANT the browser polls ThingSpeak every 20 seconds. This is simple and reliable, at the cost of a short delay (up to 20 seconds between a new reading and it appearing on screen).

4.5 The Single-Page Application (SPA)

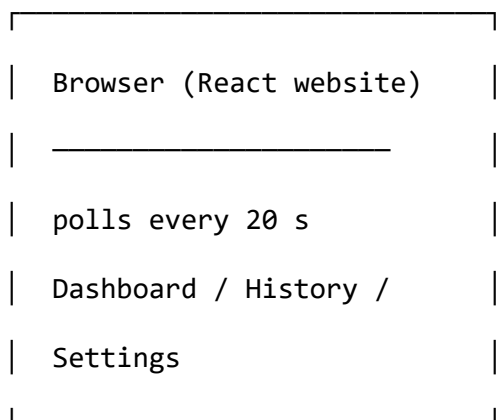
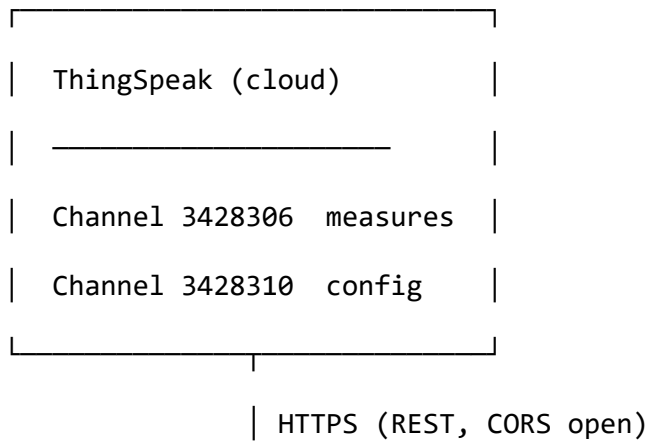
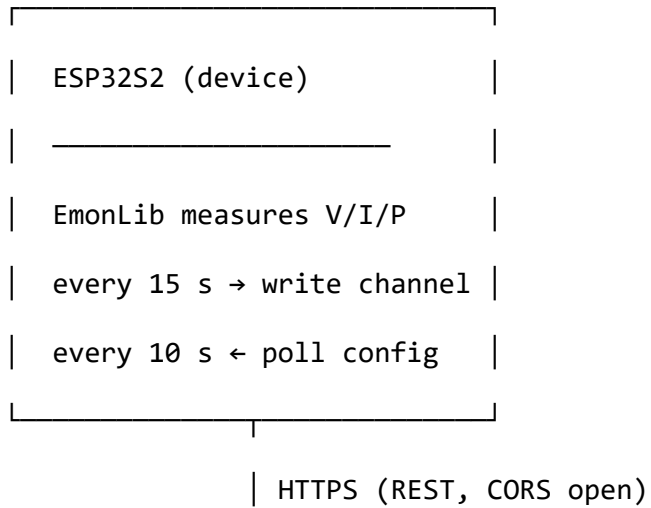
INSTANT is a **single-page application**. Instead of loading a new HTML page for each screen (Dashboard, History, Settings), the browser loads one page once and JavaScript swaps the visible content instantly. This makes navigation feel fast and app-like. The trade-off is that JavaScript must be running — that is why the HTML contains the message *“You need to enable JavaScript to run INSTANT.”*

4.6 Static Hosting

A static website is just files (HTML, CSS, JavaScript) served as-is by a web server — there is no code running on the server. INSTANT is served by **GitHub Pages**, which is free static hosting. Static hosting plus CORS-open ThingSpeak is what makes the whole project cost nothing to run.

5. System Architecture at a Glance

The architecture is deliberately simple: **device** → **cloud** → **browser**, with no server of our own in the middle.



5.1 The Two Cloud Channels

ThingSpeak stores data in **channels**, each with up to eight **fields**. INSTANT uses two channels with a clear division of responsibility:

Measurement channel 3428306 — written by the device, read by the website:

Field	Meaning	Written when
field1	Voltage (V)	every 15 s
field2	Current (A)	every 15 s
field3	Power (W)	every 15 s
field4	Relay state (1 = ON, 0 = OFF)	on change

Config channel 3428310 — written by the website, read by the device:

Field	Meaning	Written when
field1	vMax (V)	Settings save
field2	iMax (A)	Settings save
field3	pMax (W)	Settings save
field4	Relay command (1 = ON, 0 = OFF)	relay click

Think of the measurement channel as the **device's diary** and the config channel as the **control panel** that the device checks regularly.

5.2 The Free-Tier Constraint That Shapes Everything

ThingSpeak’s free plan allows a channel to be **written at most once every 15 seconds**. The website respects this by enforcing a **16-second cooldown** before any write (a little margin on top of the 15 seconds). Reads are not rate-limited, which is why the browser can poll every 20 seconds without problems. Many of the details in the code exist specifically to handle this 15-second rule gracefully, as you will see in Chapter 19.

5.3 Why GitHub Pages + ThingSpeak Works

The magic of this architecture is that everything is **public and CORS-open**. Because ThingSpeak allows any origin to read and write, the website needs no server. And because GitHub Pages serves static files from a repository, a “deploy” is just copying files to the hosting service. The result is a fully working IoT dashboard with zero servers to rent and zero software to run at night.

6. Hardware Components

This chapter describes the physical parts. Even if you never touch the hardware, understanding it helps you read the code comments (which mention pins, sensors, and calibration values).

6.1 The Microcontroller — ESP32S2

The **ESP32S2** is a small, inexpensive computer on a chip made by Espressif. It has:

- a processor (a CPU that runs programs),

- memory,
- Wi-Fi (so it can reach the internet),
- **analog-to-digital converters** (ADCs) that turn continuous voltage signals into numbers a program can use.

The firmware is written in C++ using the Arduino framework. The chip reads the sensor pins, does the maths (using a library called **EmonLib**), writes results to ThingSpeak over HTTPS, and polls ThingSpeak for commands.

Two analog pins are used:

Pin	Purpose
-----	---------

34	Voltage sensor (ZMPT101B)
----	---------------------------

35	Current sensor (SCT-013)
----	--------------------------

And one digital pin controls the relay:

Pin	Purpose
-----	---------

18	Relay (and/or an LED for testing)
----	-----------------------------------

6.2 Voltage Sensor — ZMPT101B

The ZMPT101B is a small voltage transformer. It steps the dangerous mains voltage down to a small, safe signal (a few volts) that the ADC can measure. The firmware multiplies the raw reading by a calibration constant (`V_CAL`) to convert it into real volts. Different power networks need slightly different calibration values, which is why it is a configurable number in the code.

6.3 Current Sensor — SCT-013

The SCT-013 is a **split-core current transformer** — a clamp that clips around a single wire without breaking the circuit. It produces a small signal proportional to the current flowing through the wire. The firmware multiplies the reading by an `I_CAL` calibration constant to get amps.

Because both sensors are galvanically isolated, the measurement is safe to install and does not interfere with the circuit being measured.

6.4 The Relay

A **relay** is an electrically operated switch. A small current from the microcontroller energizes a coil, which opens or closes a separate, high-power circuit. INSTANT uses the relay to switch a load on or off. The code has a setting called `RELAY_ACTIVE_LOW` that tells the firmware whether the relay module turns on with a high or a low pin signal — a common gotcha with cheap relay modules (some are “active low”, meaning `LOW = ON`).

6.5 The Measurement Math — EmonLib

The **EmonLib** library (from OpenEnergyMonitor) implements the classic “Arduino energy monitoring” algorithm:

1. Sample the voltage and current signals many times per second (e.g., 20 samples per cycle).
2. Compute the **RMS** voltage and RMS current from those samples.
3. Compute **real power** (the average of voltage $\times$ current over time).

4. Compute **power factor** as real power divided by apparent power.

The device feeds these results to ThingSpeak. Note that the website re-derives power factor itself ($P/(V \cdot I)$) rather than trusting a value sent over the wire, which keeps the cloud contract simple.

7. The ThingSpeak Cloud Platform

ThingSpeak is a free IoT analytics service. You create a channel, give it a name and description, and then either your device (or a web page) can write data to it via REST calls. ThingSpeak stores a history of values with timestamps, and lets you fetch the latest value or a window of history.

7.1 The Endpoints Used by INSTANT

All requests go to `https://api.thingspeak.com`. Each channel needs an **API key**:

- The measurement channel needs a **read key** so the browser may fetch its data.
- The config channel needs a **read key** (browser reads it) and a **write key** (browser writes to it).

Use	Method & URL	Key
Latest measurement	GET /channels/3428306/feeds/last.json?api_key=...	read

Use	Method & URL	Key
History window	GET /channels/3428306/feeds.json?minutes=N&api_key=...	read
Latest config	GET /channels/3428310/feeds/last.json?api_key=...	read
Write config/relay	POST /update with field1..field4 and api_key=...	write

7.2 Reading the Latest Value

feeds/last.json returns a JSON object describing the most recent entry, for example:

```
{
  "created_at": "2026-08-15T10:30:00Z",
  "entry_id": 1234,
  "field1": "230.4",
  "field2": "1.52",
  "field3": "350.2",
  "field4": "0"
}
```

Notice that ThingSpeak sends field values **as text strings**, even though they are numbers.

The code converts them with `parseFloat` (see Chapter 14).

7.3 Reading History

`feeds.json?minutes=N` returns up to N minutes of stored history. ThingSpeak downsamples old data, so a 7-day request returns fewer points than a 1-hour request. The browser passes the `minutes` parameter to choose the range.

7.4 Writing

Writing is a `POST /update` with a form-encoded body containing the channel's write key plus any of `field1-field8`. The response is the entry number if the write succeeded, or `0`/an error if it was rejected — which is exactly what happens when you try to write twice within 15 seconds.

7.5 The 15-Second Rule in Practice

Because the free tier rejects rapid writes, the code treats every write as precious:

- The relay click sets a cooldown timer (16 s) before the next write is allowed.
- The Settings “save” uses the same cooldown.
- If a write is rejected, the UI re-reads the actual stored value instead of guessing.

This rule is arguably the most important operational constraint in the whole project, and it is referenced throughout the code comments.

8. How the Website Is Operated (User Guide)

This chapter is the “user manual”. If you open the website, here is what you see and how to use it.

8.1 The Header

Across the top there is a bar with:

- The **INSTANT** logo and name.
- **Navigation links:** Dashboard, History, Settings.
- A **status dot** and label showing **Live** (green) when the newest reading is less than 3 minutes old, or **Offline** (red) otherwise.
- A **theme button** (sun/moon) to switch between dark and light mode. The choice is remembered on your device.

8.2 The Dashboard

The Dashboard is the home screen. It shows:

- **Four metric cards**, each with a radial gauge:
 - Voltage (V),
 - Current (A),
 - Power (W),
 - Power Factor (0–1). The numbers animate smoothly when they change (the “gliding” effect), and the gauge fills up as a value approaches its maximum.
- **A chart** titled “Power consumption — last 5 minutes”, plotting power (left axis) and voltage (right axis) over a rolling five-minute window.
- **The relay switch** — a large toggle. Clicking it sends an ON/OFF command to the device (more detail in Chapter 18).

- A “**Last update**” pill showing the time of the newest reading.
- If a safety limit is exceeded, a red **FAULT DETECTED** banner appears with a Reset button.

8.3 The History Page

The History page answers “what happened before now?”:

- **Range buttons:** 1 hour, 6 hours, 24 hours, 7 days. Selecting one fetches that window from ThingSpeak and redraws the chart.
- **A large chart** of power and voltage over the selected window.
- **Recent readings table** — the newest 12 readings with time, V, A, W, and PF.
- **Relay activity table** — the last 10 relay state changes with timestamps, derived by watching field4 change over time.

8.4 The Settings Page

Settings manages the safety thresholds and shows where the app is connected:

- **Safety thresholds:** voltage max (V), current max (A), power max (W). A badge says “From ThingSpeak” to show the values are coming from the cloud, not from local defaults.
- **Relay default state:** ON or OFF, which is published along with the thresholds.
- **“Publish settings to ThingSpeak”** button — writes all four fields to the config channel.

- **Android App** card — a **Download APK** button that downloads the INSTANT Android app (see Chapter 22).

8.5 Toasts

Small notification pop-ups (bottom-right corner) confirm actions: a green toast for success, red for errors, cyan for informational messages. They disappear automatically after a few seconds and can be dismissed with the x.

8.6 What “Live” and “Offline” Really Mean

“Live” does **not** mean the device is connected — the browser cannot know that. It means: *the newest reading in ThingSpeak is fresh (less than 3 minutes old)*. If the device stopped writing, the reading becomes stale, the status turns **Offline**, and the dashboard keeps showing the last known data rather than inventing zeros. This honesty is a deliberate design choice.

9. The React Basics You Need to Read the Code

The website is written in **React**, a popular JavaScript library for building user interfaces. If you have never seen React, here are the five ideas you must understand before reading the chapters that follow.

9.1 Components

A **component** is a reusable piece of UI built from JavaScript. In this project components are **functions** that return a description of what to draw. For example, `StatusDot` is a function

that returns a small colored circle. Components can receive **props** (short for “properties”) — arguments that customize them. `StatusDot on={online}` receives a prop named `on` telling it whether to be green or red.

9.2 JSX

JSX is the syntax React uses to write HTML-looking markup inside JavaScript. This snippet:

```
<span>Live</span>
```

is actually JavaScript that creates a `span` element with the text “Live”. JSX lets the code describe the screen next to the logic, which is why this codebase is easy to follow.

9.3 State

State is data that changes over time and triggers a redraw when it changes. React components declare state with `useState`, and the whole application shares some state through a **context** (see Chapter 13). When state changes, React re-runs the component functions and updates the screen efficiently.

9.4 Hooks

Hooks are special functions that let components remember things and react to events:

- `useState(initial)` — remember a value that can change.
- `useEffect(fn, deps)` — run code after render (e.g., start a timer, fetch data) and re-run when `deps` change.
- `useRef(initial)` — remember a value that survives re-renders without triggering redraws.

- `useCallback(fn, deps)` — remember a function so it is not recreated every render.
- `useContext(...)` — read shared state.

The project also defines a custom hook, `useAnimatedNumber`, which returns a number that smoothly eases toward a target.

9.5 Routing

Routing decides which screen is visible. INSTANT uses `react-router-dom` with a `HashRouter`: the URL contains a `#` (for example `.../Instant/#/history`). The hash makes the app work on static hosting without server-side routing rules.

9.6 Styling with styled-components

Instead of separate CSS files, the project uses **styled-components**: you write CSS inside the JavaScript using template literals, and the library generates real CSS at runtime. A styled component is a component with styles baked in:

```
const Dot = styled.span`  
  width: 10px;  
  border-radius: 50%;  
  background: ${p => (p.$on ? 'green' : 'red')};  
`;
```

Notice the syntax `${p => ...}` — a function that receives the component's props and returns the CSS value. This is how the theme colors (Chapter 17) are injected everywhere.

10. Project Structure and File Map

Here is the layout of the repository. Knowing where everything lives makes the next chapters much easier.

instant/

```
├─ .github/workflows/deploy.yml    # auto-deploys the site to GitHub Pages
├─ README.md                      # project documentation
├─ docs/INSTANT_Report.md         # this report
├─ frontend/                     # the website
│   ├─ package.json               # dependencies and scripts
│   └─ public/
│       ├─ index.html             # the single HTML page
│       ├─ manifest.json          # PWA metadata (name, icons)
│       ├─ favicon.svg            # lightning-bolt logo
│       └─ icons/                 # app icons (192px, 512px PNG)
│           └─ apk/instant.apk    # the Android app download
│               └─ .well-known/assetlinks.json # Android app-identity proof
└─ src/
    ├─ index.js                   # entry point
    ├─ index.css                  # small global CSS
    ├─ App.js                     # routes + theme shell
    ├─ theme.js                   # dark/light palettes + global styles
    ├─ api/client.js              # ThingSpeak REST wrapper
    ├─ context/AppContext.js      # global state + polling engine
    └─ hooks/useAnimatedNumber.js
```



```

|       |— components/           # Header, StatusDot, MetricCard, Gauge,
|       |                       # AppLineChart, RelaySwitch,
FaultBanner,
|       |                       # Toast, Icons
|       |— pages/               # Dashboard, History, Settings
|— android/twa/                 # Android TWA project (built locally)
|— firmware/INSTANT_ESP32S2/
    |— INSTANT_ESP32S2.ino      # device firmware (legacy MQTT version)

```

The dependency list in `frontend/package.json` is small and worth memorizing:

Package	What it provides
<code>react / react-dom</code>	The core React library and the DOM renderer
<code>react-router-dom</code>	Client-side routing
<code>styled-components</code>	CSS-in-JS styling
<code>chart.js + react-chartjs-2</code>	The line charts
<code>react-scripts</code>	The build tooling (Create React App)

11. Application Startup — `index.js` and `index.html`

11.1 The HTML Entry Point

`frontend/public/index.html` is the only real HTML file. It contains:

- metadata (charset, viewport, theme color, description),

- the app icon links (favicon.svg, the PNG icons, and the PWA manifest.json),
- fonts (Inter for body text, Orbitron for headings),
- a <title>INSTANT · Energy Monitoring</title>,
- an empty <div id="root"></div>,
- a <noscript> warning for users who disable JavaScript.

Everything the user sees is drawn *inside* #root by JavaScript. The browser never navigates to other pages; routing happens in memory.

11.2 The JavaScript Entry Point

frontend/src/index.js is the first JavaScript file the browser runs. It does four things:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { HashRouter } from 'react-router-dom';
import './index.css';
import App from './App';
import { AppProvider } from './context/AppContext';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <HashRouter>
      <AppProvider>
        <App />
```

```
    </AuthProvider>

    </HashRouter>

  </React.StrictMode>
);
```

Reading from the inside out:

1. `<App />` is the main application.
2. `<AuthProvider>` wraps it with the global state context (Chapter 13), so every component can read shared data.
3. `<HashRouter>` enables the `#/...` routing.
4. `<React.StrictMode>` is a development helper that double-runs effects to catch bugs.

`createRoot(...).render(...)` is the modern React 18 way to put a component tree into the `#root` element.

The order of the providers matters: the router must be *outside* the app so components can use routing, and the context must wrap everything that needs the shared state.

12. The App Shell — `App.js`

`App.js` is the skeleton of the whole site. It is short, so it is a good first file to study.

```
export default function App() {
  const { theme } = useApp();

  return (
```

```

<ThemeProvider theme={palettes[theme]}>

  <GlobalStyle />

  <Background />

  <Shell>

    <Header />

    <Main>

      <Routes>

        <Route path="/" element={<Dashboard />} />

        <Route path="/history" element={<History />} />

        <Route path="/settings" element={<Settings />} />

        <Route path="*" element={<Dashboard />} />

      </Routes>

    </Main>

    <Footer>INSTANT · real-time energy monitoring · ThingSpeak</Footer>

  </Shell>

  <ToastHost />

</ThemeProvider>

);
}

```

Let us dissect it:

- **ThemeProvider** comes from styled-components. It receives the current palette (palettes[theme], either dark or light) and makes it available to every styled component via the theme prop. This is the root of the whole color system.
- **GlobalStyle** (from theme.js) is a global CSS reset applied once.

- **Background** is a fixed full-screen layer with animated neon orbs behind the content.
- **Shell** centers the content with a maximum width.
- **Header** is the top navigation bar (always visible).
- **Routes / Route** connect URLs to pages:
 - `/` → Dashboard,
 - `/history` → History,
 - `/settings` → Settings,
 - `*` (anything else) → Dashboard, a sensible fallback so an unknown URL never shows a blank screen.
- **Footer** and **ToastHost** are self-explanatory.

Because every page is a different component rendered in the same shell, the header, background, and footer stay stable while only the middle section changes — that is the “single-page app” feel described in Chapter 4.

13. Global State and the Polling Engine — `AppContext.js`

This is the **brain** of the website. It holds all the data the pages display, decides when to fetch new data, and exposes the actions (toggle relay, save thresholds, etc.). It uses the **Context API**: one component (`AppProvider`) owns the state and provides it to every descendant.

13.1 Constants That Govern Behavior

At the top of the file several timing constants control everything:

Constant	Value	Meaning
FRESH_MS	3 minutes	A reading older than this makes the header show Offline
POLL_MS	20 s	How often the browser asks ThingSpeak for data
HISTORY_MS	5 min	Length of the rolling chart window on the Dashboard
HISTORY_CAP	300	Maximum number of chart points kept in memory
WRITE_COOLDOWN_MS	16 s	Minimum gap between writes (ThingSpeak allows 15 s)
RELAY_GRACE_MS	26 s	After a relay command, ignore config reads that may be stale

13.2 The Initial State

The whole application state starts as one object:

```
const initialState = {  
  voltage: 0,  
  current: 0,
```

```
power: 0,
pf: 0,
relay: 'OFF',
fault: null,
connected: false,
demoMode: false,
lastUpdated: null,
history: [],
thresholds: { vmax: 240, imax: 15, pmax: 3000 },
thresholdsSource: 'thingspeak',
relayLog: [],
relayCooldownUntil: 0,
};
```

Every field is displayed somewhere on the site: the four electrical values, the relay state, the fault message, the online flag, the timestamp, the chart history, the thresholds, where the thresholds came from, and the relay cooldown deadline.

13.3 Refs for Timely, Bug-Free Updates

Two `useRef` objects store values that callbacks need without causing re-renders:

- `stateRef` — always holds the *latest* state. Because `setState` is asynchronous, a callback reading state directly could read a stale value; reading `stateRef.current` is always current.
- `lastWriteAtRef` — the timestamp of the last write attempt. It is used to enforce the cooldown and to decide whether to trust a config read.

13.4 Notifications (notify)

`notify(type, message)` adds a toast to the list, then schedules its automatic removal after 4.5 seconds:

```
const notify = useCallback((type, message) => {  
  const id = Date.now() + Math.random();  
  setToasts((t) => [...t, { id, type, message }]);  
  setTimeout(() => setToasts((t) => t.filter((x) => x.id !== id)), 4500);  
}, []);
```

Each toast has a unique id (timestamp plus a random fraction, so two toasts in the same millisecond are still distinct). `Toast.js` renders the list; `dismissToast` removes one on demand.

13.5 The Polling Loop — the Heartbeat

`poll` is called immediately on startup and then every `POLL_MS` (20 s):

```
const poll = useCallback(async () => {  
  let latest = null;  
  let config = null;  
  try { latest = await getLatest(); } catch (_) {}  
  try { config = await getConfig(); } catch (_) {}  
  setState((s) => { ... });  
}, []);
```


Two requests run (the measurement channel and the config channel), each guarded so that a failure of one does not break the other. Then a single `setState` merges everything. Let us look at the merging logic in detail, because it encodes several important rules.

13.5.1 Thresholds

```
const thresholds = config
  ? { vmax: config.vmax, imax: config.imax, pmax: config.pmax }
  : s.thresholds;
```

If the config read worked, use the cloud thresholds; otherwise keep the previous ones. This is how the Settings page “From ThingSpeak” badge becomes true.

13.5.2 Online/Offline

```
const connected = !! (latest && Date.now() - latest.t < FRESH_MS);
```

The app is “connected” only if there *is* a latest reading **and** it is fresh. This is the honest status logic from Chapter 8.

13.5.3 Relay With a Grace Window

```
const recentlyCommanded = Date.now() - lastWriteAtRef.current <
RELAY_GRACE_MS;
const relay = config && !recentlyCommanded ? config.relay : s.relay;
```

Right after the user sends a relay command, ThingSpeak may still report the old value (the write and the read can race). For 26 seconds the app therefore **trusts its own last value** instead of the config read, preventing the UI from “snapping back” to the old state.

13.5.4 History Update

```
const point = { t: latest.t, voltage: latest.voltage, current:
latest.current, power: latest.power, pf: latest.pf };
const last = s.history.length ? s.history[s.history.length - 1] : null;
const history =
  last && last.t === latest.t
    ? s.history
    : [...s.history, point]
      .filter((p) => p.t > Date.now() - HISTORY_MS)
      .slice(-HISTORY_CAP);
```

The new reading is appended to the rolling history **only if it is different from the last one** (deduplication). Old points (older than 5 minutes) are filtered out, and the list is trimmed to 300 points. This keeps the Dashboard chart a tidy, sliding five-minute window.

13.5.5 The Fault Check

```
const exceeded =
  latest.voltage > thresholds.vmax ||
  latest.current > thresholds.imax ||
  latest.power > thresholds.pmax;
```

If any live reading exceeds its threshold, `fault` is set to the message 'Safety limit exceeded', which makes the red banner appear on the Dashboard.

13.6 toggleRelay — the Cooldown-Protected Write

This is the most subtle function in the app. Read it step by step:

```

const toggleRelay = useCallback(async () => {
  const now = Date.now();
  const since = now - lastWriteAtRef.current;
  if (since < WRITE_COOLDOWN_MS) {
    notify('info', `ThingSpeak allows one config update per 15 s – retry in
${...} s`);
    return;
  }

  const next = stateRef.current.relay === 'ON' ? 'OFF' : 'ON';
  lastWriteAtRef.current = now;
  setState((s) => ({ ...s, relay: next, relayCooldownUntil: now +
WRITE_COOLDOWN_MS }));

  try {
    await saveConfig({ vmax: ..., imax: ..., pmax: ..., relay: next });
    setState((s) => ({ ...s, relay: next }));
    notify('success', `Relay command sent: ${next}`);
  } catch (err) {
    let actual = next;
    try { actual = (await getConfig()).relay; } catch (_) {}
    setState((s) => ({ ...s, relay: actual }));
    notify('error', `${err.message} (1 update / 15 s limit)`);
  }
}, [notify]);

```

The logic:

1. **Cooldown check:** if less than 16 s since the last write, inform the user and stop. This prevents the free-tier rejection entirely (a “first line of defense”).
2. **Compute the target:** flip the current relay state (ON ↔ OFF).
3. **Optimistic update:** record the write time and immediately flip the UI. The user gets instant feedback; we do not wait for the cloud.
4. **Write:** POST the command (plus the current thresholds, because the write overwrites *all* fields of the channel) to ThingSpeak.
5. **Success:** keep the optimistic value and show a green toast.
6. **Failure:** never guess. Re-read the config channel to learn the real stored value, show it, and display a red toast. If even the re-read fails, keep the optimistic value.

This function is the direct answer to a real bug: without the cooldown and the grace window, rapid clicks caused the switch to “snap back” to the previous position. Chapter 19 examines the bug story.

13.7 `saveThresholds` — the Settings Writer

`saveThresholds(payload)` follows the same pattern:

- Enforce the cooldown (returns `false` so the page can stop its spinner).
- Write all four fields (`vMax`, `iMax`, `pMax`, `relay`) to the config channel.
- On success, update thresholds and relay in state, set the cooldown, toast success, return `true`.

- On failure, toast the error and return false.

Note that it writes the *relay* too. Because each channel update replaces the whole entry, every write must carry all four fields so nothing is accidentally lost. That is why `toggleRelay` also sends the thresholds and `saveThresholds` also sends the relay state.

13.8 Other Actions

- `resetFault()` — clears the fault message locally (the banner disappears until the next violating reading arrives).
- `toggleTheme()` — switches dark/light and saves the choice to `localStorage` under `instant-theme`, so the preference survives a page reload.

13.9 The Context Value

At the bottom, `AppProvider` bundles everything into one value object:

```
const value = {
  state, connected, theme, toasts,
  notify, dismissToast, toggleRelay, saveThresholds, resetFault, toggleTheme,
};
return <AppContext.Provider value={value}>{children}</AppContext.Provider>;
```

Any component can call `useApp()` to get this object. `useApp` also throws a helpful error if it is used outside the provider — a common mistake that would otherwise produce a confusing crash.

14. Talking to ThingSpeak — `api/client.js`

`client.js` is a small, clean “wrapper” around the ThingSpeak REST API. All the messy URL building, key handling, JSON parsing, and number conversion lives in one place so the rest of the app never deals with HTTP details.

14.1 Configuration Object

```
const TS = {  
  base: 'https://api.thingspeak.com',  
  channel: '3428306',           // measurements  
  readKey: 'UIWSRR7X029RCD5V',  
  configChannel: '3428310',    // config  
  configReadKey: 'F54BNJ6PACIS30KD',  
  configWriteKey: 'K7TDWWD0WEQN97RR',  
};
```

All channel IDs and keys are constants at the top. This is the single place you would edit to point the app at a different ThingSpeak account.

14.2 Number Conversion — `num()`

```
function num(value) {  
  const n = parseFloat(value);  
  return Number.isFinite(n) ? n : 0;  
}
```

ThingSpeak sends numbers as strings; `parseFloat` converts them. If the value is missing or not a number, the function safely returns 0 instead of NaN (Not-a-Number), which would otherwise poison charts and calculations.

14.3 Normalizing a Reading — `parseReading()`

```
function parseReading(f) {  
  if (!f || f.field1 == null || f.field2 == null) return null;  
  const voltage = num(f.field1);  
  const current = num(f.field2);  
  const power = num(f.field3);  
  const relay = String(f.field4) === '1' ? 'ON' : 'OFF';  
  const pf = voltage * current > 0 ? Math.max(0, Math.min(1, power / (voltage  
* current))) : 0;  
  return {  
    t: Date.parse(f.created_at) || Date.now(),  
    voltage, current, power, pf, relay, lastUpdated: f.created_at,  
  };  
}
```

This converts one raw ThingSpeak entry into the app's internal shape:

- Fields are converted to numbers.
- Field4 ("1"/"0") becomes 'ON'/'OFF'.
- Power factor is derived as `power / (voltage × current)`, clamped between 0 and 1, and guarded against division by zero.

- A clean timestamp t (milliseconds) is computed from the `created_at` text.

If the entry is empty or missing the essential fields, it returns `null` so callers can skip it.

14.4 The Fetch Helper — `getJSON()`

```
async function getJson(url) {
  const res = await fetch(url, { headers: { Accept: 'application/json' } });
  if (!res.ok) throw new Error(`ThingSpeak error (${res.status})`);
  return res.json();
}
```

`fetch` is the browser's built-in HTTP function. If the server returns an error status, an exception is thrown with a readable message; otherwise the response body is parsed as JSON.

14.5 The Four Public Functions

`getLatest()` — fetches `feeds/last.json` and normalizes it. This is the “current reading” used by the dashboard.

`getHistory(minutes)` — fetches `feeds.json?minutes=N`, maps every entry through `parseReading`, and drops invalid ones. It also builds a **relay log** by scanning `field4` for changes:

```
let prev = null;
for (const f of feeds) {
  const st = String(f.field4) === '1' ? 'ON' : 'OFF';
  if (prev !== null && st !== prev) relayLog.push({ t:
Date.parse(f.created_at), state: st });
}
```



```
    prev = st;
}
```

Each time the relay state differs from the previous entry, a change record is created. The History page shows the last 10 of these.

getConfig() — fetches the latest config entry and returns { vmax, imax, pmax, relay }.

Note the fallbacks: if a threshold field is empty, it defaults to 240, 15, 3000 — matching the app’s initial state.

saveConfig({ vmax, imax, pmax, relay }) — writes with a POST:

```
const body = new URLSearchParams({
  api_key: TS.configWriteKey,
  field1: String(vmax), field2: String(imax),
  field3: String(pmax), field4: relay === 'ON' ? '1' : '0',
});

const res = await fetch(`${TS.base}/update`, {
  method: 'POST', body,
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
});

if (!res.ok) throw new Error(`ThingSpeak write failed (${res.status})`);
```

URLSearchParams correctly encodes the body. A non-OK response throws, and the caller (AppContext) handles the error with a toast. This is where the 15-second limit surfaces as an HTTP error.

15. Reusable UI Components

The components/ folder contains small, focused pieces. This chapter explains each one and the ideas behind it.

15.1 StatusDot

The simplest component: a 10 px circle that is green with a pulse animation when on is true, red otherwise. It proves the pattern *props in → styled output*.

15.2 Header

The navigation bar. Highlights:

- Uses NavLink from react-router-dom so the active link gets a special style automatically.
- Shows the logo in a gradient square, the INSTANT name, three nav links, the live/offline status (StatusDot + text), and the theme toggle.
- `const online = connected && state.connected;` — both the context value and the state field are used (they are the same, but this shows defensive coding).

15.3 MetricCard and Gauge

A **MetricCard** is one of the four dashboard cards. It receives props: label, value, unit, color, icon, max, and decimals.

Inside, the value passes through the custom hook useAnimatedNumber (see below) so it glides toward the target. The card lays out:

1. A header row: colored icon tile + label.
2. A gauge box with a **Gauge** underneath and the number **on top** of it.

The **Gauge** component draws two circles with SVG:

- a faint background ring,
- a colored “progress” ring using `strokeDasharray` and `strokeDashoffset`.

The math: the full circumference is $2\pi r$. The fraction filled is `value/max` (clamped to 0–1).

The dash offset `circumference × (1 - fraction)` hides the unfilled part. A CSS transition on `stroke-dashoffset` makes the ring animate smoothly when the value changes.

```
const pct = Math.min(1, Math.max(0, value / max));
const offset = circumference * (1 - pct);
```

15.4 `useAnimatedNumber` — the Gliding Number Hook

A custom hook that smoothly eases a displayed number toward a target over 700 ms using `requestAnimationFrame` and a cubic ease-out curve:

```
const tick = (now) => {
  const t = Math.min(1, (now - start) / duration);
  const eased = 1 - Math.pow(1 - t, 3);
  const value = from + (target - from) * eased;
  currentRef.current = value;
  setDisplay(value);
  if (t < 1) raf = requestAnimationFrame(tick);
};
```

- `eased` follows the curve $1 - (1 - t)^3$, which starts fast and finishes gently.

- The current value is stored in a ref so the next animation starts from where the last one ended (no jumps).
- When the target changes, the effect re-runs; when it is done, the animation frame is cancelled to avoid leaks.

This is a great example of a small, reusable piece of polish that makes the dashboard feel alive.

15.5 AppLineChart

A wrapper around Chart.js that all charts share. It receives `labels` (x-axis), `series` (one or more datasets), a height, and axis titles.

Notable details:

- Registers only the Chart.js modules used, keeping the bundle small.
- **Gradient fills:** for filled series, a vertical gradient is built from the chart's drawing area (stronger color at the top, transparent at the bottom).
- **Theme awareness:** tick and tooltip colors come from `useTheme()`, so the chart automatically matches dark or light mode.
- **Two y-axes:** power uses the left axis (`y`), voltage uses the right axis (`y1`), because their scales are completely different.
- `pointRadius: 0` draws no dots (cleaner for dense data); `tension: 0.35` curves the lines.

15.6 RelaySwitch

The big glowing toggle. This component's behavior is tightly coupled to AppContext:

```
const locked = state.relayCooldownUntil > Date.now();

const handle = async () => {
  if (busy || locked) return;
  setBusy(true);
  await toggleRelay();
  setBusy(false);
};
```

- busy is local UI state: while a click is in flight, a spinner replaces the icon and clicks are ignored.
- locked is true while the cooldown is active; the button is disabled and shows reduced opacity.
- The knob slides left/right with a springy cubic-bezier transition; the green glow pulses when ON.
- The status line under the switch shows SWITCHING while busy, otherwise RELAY ON or RELAY OFF.
- A useEffect with a 1-second interval forces a re-render every second so the locked state resolves promptly after the cooldown ends.
- Accessibility: role="switch", aria-checked, and an aria-label.

15.7 FaultBanner

A red gradient banner with a warning icon, the text **FAULT DETECTED**, a description, and a **Reset** button. It is only rendered when `fault` is truthy (see Dashboard). The Reset button calls `resetFault`.

15.8 Toast

Renders the toast list from `AppContext` in a fixed container at the bottom-right. Each toast is colored by type (success green, error red, info cyan) via a left border, slides in, and can be dismissed with the x button.

15.9 Icons

A tiny icon library of inline SVG components (bolt, wave, zap, pulse, warning, sun, moon, plug). All icons share a common `Svg` wrapper that sets size, stroke, and stroke width, so icons are just a few path lines each. `LogoIcon` is simply `BoltIcon` — the lightning bolt is the brand mark.

16. The Pages

16.1 Dashboard — `pages/Dashboard.js`

The Dashboard assembles the components into the home screen. Key logic:

Color coding per metric:

```
const ACCENTS = {  
  voltage: '#00CFFF',
```

```
    current: '#FF9F2E',  
    power: '#A96BFF',  
    pf: '#00E5A0',  
  };
```

Chart data construction. The rolling history from state is turned into Chart.js inputs:

```
const labels = history.map((h) => new Date(h.t).toLocaleTimeString([], {  
  hour12: false }));  
const series = [  
  { label: 'Power (W)', data: history.map((h) => h.power), color:  
    ACCENTS.power, fill: true, borderWidth: 2 },  
  { label: 'Voltage (V)', data: history.map((h) => h.voltage), color:  
    ACCENTS.voltage, fill: false, borderWidth: 1.5, yAxisID: 'y1' },  
];
```

Gauge ranges. Each MetricCard picks a sensible maximum so the gauge is meaningful:

- Voltage: $\text{Math.max}(260, \text{thresholds.vmax} * 1.1)$ — a bit above the safety limit.
- Current: $\text{Math.max}(15, \text{thresholds.imax} * 1.5)$.
- Power: $\text{Math.max}(2500, \text{thresholds.pmax})$.
- Power factor: always 1.

Layout. Four cards in a responsive grid; below them, the chart (2/3 width) and the relay switch (1/3 width). The fault banner appears between the header row and the cards when present.

16.2 History — pages/History.js

The History page manages its **own** state (it does not need global state):

```
const [range, setRange] = useState(1440);
const [data, setData] = useState(null);
const [loading, setLoading] = useState(false);
```

- range — the selected window in minutes (defaults to 24 hours = 1440).
- data — the fetched { points, relayLog }.
- loading — whether a fetch is in flight.

The **fetch effect** is the interesting part:

```
useEffect(() => {
  let active = true;
  setLoading(true);
  getHistory(range)
    .then((res) => { if (active) setData(res); })
    .catch((err) => { if (active) notify('error', err.message || 'Failed to
load history'); })
    .finally(() => { if (active) setLoading(false); });
  return () => { active = false; };
}, [range, notify]);
```

- active is a flag flipped to false when the component unmounts or the range changes. This prevents a slow, outdated response from overwriting a newer one (the classic **race condition** guard).

- The effect re-runs whenever range changes, so clicking “7 days” automatically fetches the new window.
- Failures become error toasts rather than blank screens.

Data shaping. The fetched points become chart labels (full date + time for history) and two series. The **recent readings** table shows the last 12 points reversed (newest first). The **relay activity** table shows the last 10 relay log entries, with colored ON/OFF badges.

Empty states. When there is no data yet, the page says so (“No history yet — waiting for data from ThingSpeak.”) instead of drawing an empty chart — a nice touch for beginners to notice.

16.3 Settings — `pages/Settings.js`

The Settings page is a form. Key ideas:

Form state. Three text inputs (vMax, iMax, pMax) plus a relay selector, all as `useState` strings. A `useEffect` keeps the form synced to the loaded state:

```
useEffect(() => {
  const th = state.thresholds;
  if (th.vmax) {
    setVmax(String(th.vmax));
    setImax(String(th.imax));
    setPmax(String(th.pmax));
  }
  setRelayDefault(state.relay);
}, [state.thresholds, state.relay]);
```

Validation before writing. `validate()` checks the numeric ranges (voltage 1–400, current 0.1–100, power 1–100000) and returns an error message if invalid. `handleSave` shows the error as a toast and refuses to write — bad values never reach the cloud.

The save. `saveThresholds` from context does the write and returns `true/false`; the button shows “Publishing...” while waiting.

Info panels. The Note texts are mini-documentation: which channel fields each value maps to, the REST endpoint, and the polling cadence. The **Android App** card (with the Download APK button) is described in Chapter 22.

17. Theming and Styling

17.1 Two Palettes

`theme.js` defines a `palettes` object with a dark and a light palette. Each palette is a set of named colors:

Key	Dark value	Light value	Used for
<code>bg</code>	<code>#0B0D17</code>	<code>#F1F3FA</code>	page background
<code>bg2</code>	<code>#141830</code>	<code>#FFFFFF</code>	raised surfaces / tooltips
<code>text</code>	<code>#E9ECFF</code>	<code>#151A33</code>	main text
<code>textDim</code>	<code>#8A91B4</code>	<code>#5A6288</code>	secondary text
<code>cyan</code>	<code>#00D2FF</code>	<code>#00A2C9</code>	brand / links

Key	Dark value	Light value	Used for
magenta	#FF007F	#E60077	brand accent
green	#00E5A0	#00B080	success / relay ON
red	#FF3B6B	#E02D55	errors / faults
amber	#FFD166	#C99A1E	warnings
...	...	...	...

Because every styled component reads colors through `(p) => p.theme.color`, switching the palette re-colors the **entire** site in one place.

17.2 Global Styles

`GlobalStyle` (via `createGlobalStyle`) applies CSS that is not component-specific:

- `box-sizing`: `border-box` for predictable sizing,
- font families (Inter for text, Orbitron for headings),
- the background/color with a smooth transition,
- keyframe animations `slideIn` and `pulseDot` used by toasts and status dots.

17.3 The Animated Background

Background is a fixed, full-screen div behind everything. It uses two radial gradients (magenta top-right, cyan bottom-left) plus two blurred, drifting “orbs” animated with a `slow drift` keyframe. All the visual drama of the site comes from CSS — no images required.

17.4 The Global CSS File

`index.css` is deliberately tiny: a color-scheme: dark hint, a body margin reset, and styled scrollbars. The comment says it plainly: “the real theming lives in styled-components GlobalStyle.”

18. End-to-End Feature Walkthroughs

This chapter traces real user actions through the entire stack, tying together every chapter before it.

18.1 “Turn the relay OFF”

What the user does: On the Dashboard, clicks the big relay toggle.

What happens, step by step:

1. RelaySwitch’s `handle()` runs. `busy` becomes `true` (spinner shows).
2. `toggleRelay()` in `AppContext` checks the cooldown. If less than 16 s since the last write, it shows “retry in N s” and stops.
3. The cooldown is passed; the UI **optimistically** flips to OFF (`setState({ relay: 'OFF' })`) and records `relayCooldownUntil`.
4. `saveConfig({ vmax, imax, pmax, relay: 'OFF' })` in `client.js` builds a form body and POSTs to `api.thingspeak.com/update` with the config channel’s write key.
5. ThingSpeak stores the entry. The device’s next 10-second poll reads `field4 = 0`.

6. The device firmware applies the command, switches the relay off, and writes its actual state to the **measurement** channel (`field4 = 0`).
7. Meanwhile the browser continues polling every 20 s. For 20 s after the write, the poll ignores the config read for the relay (grace window) so the OFF state is not overwritten by a stale read.
8. After the grace window, polls trust the config read again. The next successful read confirms OFF.
9. A green toast confirms: “Relay command sent: OFF”.

Delays the user can expect: the toggle flips instantly (optimistic), the physical relay switches within ~10 s (the device’s poll interval), and the cloud-confirmed state appears within ~20 s.

18.2 “Save new thresholds”

1. On Settings, the user edits vMax/iMax/pMax and clicks “Publish settings to ThingSpeak”.
2. `validate()` checks the ranges; if invalid, an error toast appears and nothing is sent.
3. `saveThresholds` enforces the same 16 s cooldown as the relay.
4. `saveConfig` writes all four fields (including the relay, so nothing is lost).
5. On success, state thresholds update instantly, the Settings inputs re-sync, a success toast appears, and the fault banner (if any) now compares readings against the new limits on the next poll.

18.3 “Check what happened yesterday”

1. The user opens History and clicks “24 hours”.
2. `setRange(1440)` triggers the effect; `getHistory(1440)` GETs `feeds.json?minutes=1440`.
3. `client.js` maps every entry into a point and derives the relay log from field4 changes.
4. The page redraws the chart, the recent-readings table (last 12, newest first), and the relay-activity table (last 10 changes).
5. Because the data lives in ThingSpeak, the history is available even when the device is offline — it is a stored record, not a live stream.

18.4 “Why does the status say Offline?”

The header computes `connected = latest exists AND fresh (< 3 min)`. Offline appears when:

- ThingSpeak is unreachable from the browser, or
- the latest stored reading is older than 3 minutes (device powered off, Wi-Fi lost, or it stopped writing).

The site keeps showing the last known numbers (it does not zero them), and the status dot turns red. Once a fresh reading arrives, everything recovers automatically — no reload needed.

19. Error Handling and Edge Cases

A good dashboard is judged by how it behaves when things go wrong. This chapter catalogs the failure modes INSTANT handles and how.

19.1 The Rate-Limit Bug That Shaped the Design

In an earlier version of the app, clicking the relay twice quickly caused a strange bug: the switch flipped, then *snapped back* to the previous position, and a second click was needed. The cause was the ThingSpeak free-tier rule — **one write per 15 seconds per channel**.

The first write succeeded; the second was rejected with HTTP 400, and the old code reacted to the failure by reverting to the (stale) config value. The fix had three parts, all visible in the code:

1. **WRITE_COOLDOWN_MS = 16000** — refuse to write twice within 16 s, so the rejection is never triggered in the first place.
2. **RELAY_GRACE_MS = 26000** — after a command, ignore config reads for 26 s so an in-flight stale read cannot overwrite the optimistic UI.
3. **No blind reverts** — on a write failure, the app *re-reads* the actual stored value and shows the truth, rather than guessing.

This is a textbook example of designing software around a real-world constraint, and it is why the cooldown logic appears in both `toggleRelay` and `saveThresholds`.

19.2 ThingSpeak Unreachable

`getLatest/getConfig` throw on network errors. `poll` catches each request separately:

- If the latest-read fails, connected becomes false and the dashboard keeps the previous values.
- If the config-read fails, the previous thresholds are kept.

The two are independent, so one failure does not blank the whole dashboard.

19.3 Bad or Missing Data

- Missing fields → `num()` returns 0 instead of NaN.
- Entries missing essential fields → `parseReading` returns null, and callers filter those out.
- Power factor division by zero → guarded (`voltage * current > 0`), clamped to 0–1.

19.4 Empty History

History shows friendly “No history yet” messages instead of an empty chart, and the Dashboard handles a zero-length history gracefully (empty labels/series).

19.5 Form Validation

Settings refuses to publish out-of-range thresholds, so garbage never reaches the cloud or the device.

19.6 Race Conditions

- The History page’s active flag prevents stale responses from overwriting newer ones when the user switches ranges quickly.
- `stateRef` ensures async callbacks always read the current state.

- The relay grace window prevents read/write races after a command.

19.7 Unknown URLs

`<Route path="*" element={<Dashboard />} />` means an unknown URL shows the Dashboard rather than a blank page or a confusing 404.

20. Security Considerations

It is important to be honest about the security model of a project like this.

20.1 Keys in the Browser

The ThingSpeak **read and write keys are embedded in the JavaScript bundle**. Anyone can open the site, view the source, and read (or write) the channels. This is **by design** for a public demo, but it is a genuine limitation:

- Anyone can change the thresholds or toggle the relay.
- This is acceptable only because the project is a demonstration.

20.2 How a Real Deployment Would Differ

For production you would:

- Use **private** ThingSpeak channels.
- Keep write keys **server-side only** — a small backend would receive browser requests, authenticate users, and perform the writes itself, so keys never reach the browser.

- Add authentication (accounts) so only authorized people can control the relay.
- Note: even with a private channel, reads from the browser still expose the read key — the write key is the one that really matters for control.

20.3 Input Validation

All threshold inputs are validated in the UI before publishing, reducing the chance of nonsensical values reaching the device (though a malicious user could bypass the UI — see 20.1).

20.4 Physical Safety

The firmware is designed to cut the relay when readings exceed thresholds, as a last line of defense independent of the website. Web-based control is best treated as a convenience, not a safety mechanism.

21. Building, Running, and Deploying

21.1 Running Locally (Development)

With Node.js installed:

```
cd frontend
```

```
npm install      # first time only
```

```
npm start        # starts the dev server on http://localhost:3000
```

The dev server gives you hot reloading (edit a file, see the change instantly) and helpful error overlays. No environment files or keys to configure — the keys are in the source.

21.2 Building for Production

```
npm run build
```

Create React App compiles the JSX/CSS into optimized static files in `frontend/build`:

- HTML, CSS, and a bundle of JavaScript with hashed filenames (e.g. `main.c627c1e2.js`). The hash changes whenever content changes, which helps browsers cache correctly.

The build is fully portable: because the site talks to ThingSpeak directly (CORS open) and uses HashRouter plus `"homepage": "."` (relative paths), you can serve `frontend/build` from any static host — or even open `index.html` from disk.

21.3 Automatic Deployment to GitHub Pages

`.github/workflows/deploy.yml` is a GitHub Actions workflow that runs on every push to `main`:

1. Checks out the repository.
2. Installs Node 22.
3. Runs `npm ci` in `frontend` (installs exact versions from the lockfile).
4. Runs `npm run build`.
5. Uploads `frontend/build` as a Pages artifact.
6. Deploys it to GitHub Pages via `actions/deploy-pages`.

The result: pushing code to GitHub automatically publishes the new site at `https://vwoudka.github.io/Instant/` within about a minute. This is called **continuous deployment** — no manual steps.

21.4 Why the Workflow Only Builds the Frontend

The workflow builds only frontend. The device firmware is uploaded to the ESP32 with the Arduino IDE (it is not deployed through the website pipeline), and the Android app is built locally (Chapter 22). The GitHub workflow is purely the website's release pipeline.

22. The Android App (APK)

The website can also be installed as an Android app. It is a **Trusted Web Activity (TWA)**: a thin Android shell that opens the website full-screen in Chrome, showing an app icon in the launcher and behaving like a native app — but it is just the website in a window.

22.1 The Pieces

Three things make this work:

1. **A web app manifest** (`frontend/public/manifest.json`) with the app name, `start_url`, display mode, theme color, and icons (192 px and 512 px PNG). This makes the site installable as a PWA.
2. **The APK itself** (`frontend/public/apk/instant.apk`) — a ~1.2 MB signed Android package built with the Bubblewrap tool, whose manifest declares the package name `app.instant.energy` and the URL it opens.

3. **assetlinks.json** (frontend/public/.well-known/assetlinks.json) — a JSON file proving that the website legitimately owns the app. It lists the package name and the SHA-256 fingerprint of the app's signing certificate. Android checks this file when verifying the TWA, which is why it must live at a fixed path on the site.

22.2 How a User Gets It

On the Settings page, the **Download APK** button links to `./apk/instant.apk` (with a download attribute). The user downloads it to their phone, allows installing apps from unknown sources, and installs it. Because the APK is hosted by the website itself, new versions are simply a new APK at the same URL — the site is the app store.

22.3 Rebuilding the APK

The Android project lives in `android/twa` and is **git-ignored** — it is a local build toolchain (JDK 17 + Android SDK + Bubblewrap), not part of the website repository. To rebuild:

1. Ensure the JDK, Android SDK, and `@bubblewrap/cli` are installed (they were set up during the initial build).
2. Re-run the TWA generator and `bubblewrap build` from `android/twa`.
3. Copy `app-release-signed.apk` to `frontend/public/apk/instant.apk`.
4. Commit and push; the workflow redeploys the site with the new APK.

The signing keystore and its password are kept locally (not in git), so the app can be updated with the same identity.

23. The ESP32 Firmware

The repository contains the device program:

`firmware/INSTANT_ESP32S2/INSTANT_ESP32S2.ino`.

Important note for readers: the firmware file currently checked into the repository is a **legacy MQTT version** from an earlier architecture. The *current* deployed system uses the ThingSpeak architecture described throughout this report (the device writes the measurement channel and polls the config channel), and the README describes that ThingSpeak firmware. The MQTT sketch is kept for reference. The hardware theory below applies to both versions.

23.1 What the Firmware Does (ThingSpeak Architecture)

A ThingSpeak-based sketch would:

1. Connect to Wi-Fi.
2. Every 15 s (T_{SEND}): measure V/I/P with EmonLib and **write** `field1..field3` (plus relay state in `field4`) to the measurement channel.
3. Every 10 s (T_{READ}): **poll** the config channel, read `field1..field4`, and apply new thresholds or a new relay command.
4. Enforce safety: if a reading exceeds $v_{\text{Max}}/i_{\text{Max}}/p_{\text{Max}}$, force the relay off and report a fault.

The README documents the wiring table:

Item	Value
------	-------

Item	Value
Voltage sensor	ZMPT101B on pin 34
Current sensor	SCT-013 on pin 35
Relay / LED	pin 18
Calibration	V_CAL, I_CAL, PHASE
Relay polarity	RELAY_ACTIVE_LOW
Write cadence	15 s (T_SEND)
Config poll	10 s (T_READ)

23.2 Reading the Legacy MQTT Sketch

Even though it is not the current transport, the checked-in sketch teaches the hardware fundamentals:

- **Setup:** configures the relay pin to a safe state, sets ADC resolution/attenuation, configures EmonLib for the voltage and current pins, calibrates with a few empty readings, connects to Wi-Fi, and connects to the MQTT broker.
- **The loop** is time-sliced with `millis()` (no `delay()` blocking):
 - every 5 s: reconnect Wi-Fi/MQTT if needed,
 - every 1 s: take a measurement,
 - every 2 s: publish measurements,
 - continuously: `mqtt.loop()` keeps the connection alive, and `controlRelay()` applies safety logic.

- **Safety logic** (`controlRelay`): trips the relay if voltage is too high, too low (undervoltage), current too high, or power too high. After a trip, a 5-second “cool-down” must pass before a new command is accepted.
- **MQTT message handling**: subscribes to relay commands and threshold topics, updating local variables when messages arrive.

The MQTT version publishes every 2 s (MQTT has no rate limit), whereas ThingSpeak’s 15 s write limit forces the slower cadence — a perfect illustration of how the cloud platform shapes the device behavior.

24. Troubleshooting

A practical table of symptoms and fixes, drawn from the project’s own README and from the way the code is written.

Symptom	Likely cause	What to do
Blank / white page	Opening <code>public/index.html</code> directly (it has no compiled JS)	Open the built <code>frontend/build/index.html</code> , or serve the build folder
Header shows Offline	Device stopped writing, or ThingSpeak unreachable	Check the ESP is powered and Wi-Fi credentials are correct; the site will recover automatically when a fresh

Symptom	Likely cause	What to do
		reading arrives
Gauges stay at 0	No fresh feeds in the measurement channel, or bad sensor wiring/calibration	Check the channel has recent entries; verify sensor wiring and V_CAL/I_CAL
Relay toggle has no effect	Config write failed (check the toast), or the device is not running	Retry after the 16 s cooldown; confirm the firmware is polling the config channel
Relay switch seems to “snap back”	(Old behavior — now fixed) writes within 15 s are rejected	Update the site; if it still happens, watch for error toasts and wait for the cooldown
Chart is empty	Selected range has no stored feeds yet	Pick a smaller range (e.g. 1 hour)
“Retry in N s” toast on relay/Settings	The 16 s write cooldown is active	Wait a few seconds and try again
APK won’t install	“Unknown sources” is disabled	Allow installing apps from unknown sources; verify you downloaded the whole file
APK opens but shows a message	assetlinks verification failing	Confirm .well-known/assetlinks.json is

Symptom	Likely cause	What to do
instead of the site		deployed and matches the package + fingerprint

25. Glossary

Term	Definition
API	Application Programming Interface — a set of defined requests a service accepts (e.g. “give me the latest reading”).
API key	A secret string used to authenticate a request to a service.
Apparent power	$V \times I$ — the power the supply must provide, ignoring whether it is used usefully.
CORS	Cross-Origin Resource Sharing — the mechanism by which a server allows browsers from other origins to read it.
Channel	In ThingSpeak, a container of up to 8 fields of data with timestamps.
Component	A reusable piece of UI in React, written as a function returning JSX.
Context	A React feature for sharing state across many components without passing it through every level.

Term	Definition
Cooldown	A minimum delay before an action may be repeated (here, 16 s before the next write).
Current (I)	The flow of electric charge, measured in amperes (A).
Deploy	Publishing a built version of the software to the hosting service.
fetch	The browser's built-in function for making HTTP requests.
Field	In ThingSpeak, one named column of a channel's data.
Frontend	The part of an application that runs in the browser.
GitHub Actions	GitHub's automated workflow system; used here to deploy the site on every push.
GitHub Pages	GitHub's free static website hosting.
Hook	A React function (useState, useEffect, ...) that adds features to a component.
HTTP	The protocol browsers and devices use to send requests and receive responses.
IoT	Internet of Things — physical devices connected to the Internet.
JSON	JavaScript Object Notation — a text format for structured data.
JSX	The HTML-like syntax used inside React JavaScript.
Polling	Repeatedly asking a server "is there anything new?" on a timer.

Term	Definition
Power (P)	The rate of using energy, in watts (W); $P = V \times I$.
Power factor (PF)	Real power ÷ apparent power; a value from 0 to 1.
Prop	An argument passed to a React component.
React	A JavaScript library for building user interfaces from components.
Real power	The power actually doing useful work, measured by averaging $V \times I$.
Relay	An electrically operated switch.
REST	A style of HTTP API organized around resources and GET/POST verbs.
RMS	Root-mean-square — the standard measure of an AC voltage or current.
Single-page app (SPA)	A website that loads once and swaps content in the browser.
State	Data a component remembers; changes to it re-render the UI.
styled-components	A library for writing CSS inside React components.
TWA	Trusted Web Activity — an Android app shell that shows a website full-screen.
Threshold	A safety limit (v_{Max} , i_{Max} , p_{Max}) that must not be exceeded.

Term	Definition
Toast	A small transient notification pop-up.
Voltage (V)	Electrical “pressure”, measured in volts (V).

26. Further Reading

If this report made you curious, here are good next steps, roughly in order of difficulty.

For the web side: - The React official tutorial: <https://react.dev/learn> - styled-components documentation: <https://styled-components.com/docs> - Chart.js documentation: <https://www.chartjs.org/docs/> - How HTTP works (MDN): <https://developer.mozilla.org/docs/Web/HTTP>

For the IoT/cloud side: - ThingSpeak documentation: <https://docs.thingspeak.com> - ThingSpeak REST API reference: <https://www.mathworks.com/help/thingspeak/rest-api.html>

For the hardware side: - OpenEnergyMonitor’s “How to build an Arduino energy monitor”: <https://docs.openenergymonitor.org> - ESP32 Arduino core: <https://github.com/espressif/arduino-esp32>

In this repository: - `README.md` — concise project documentation including the API reference table. - The source files under `frontend/src` — now that you have read this report, the code should read like a story: `client.js` (how data comes in), `AppContext.js` (how it is kept fresh and written back), the components (how it is shown), and the pages (how it is assembled).

End of report. INSTANT — real-time energy monitoring, from sensor to screen.